To be announced *

F. A. Fortunato † J. M. Martínez †

July 1st, 2026

Abstract

In preparation
Key words: River flow modeling,

1 Introduction

2 Optimization in the presence of forbidden sets

{opfor}

Many optimization problems involve forbidden regions. This means parts of the domain where the objective function is not defined, is unreliable, or has no physical meaning. Sometimes these regions are excluded by introducing constraints to the problem, but on other occasions, resorting to constraints does not work. On the one hand, forbidden regions may not be well-characterized by constraints defined by algebraic equalities or inequalities. On the other hand, even when safe constraints can be identified, algorithms tend to hover near their infeasibility, eagerly searching for low values of the objective function, when the right move would be to abandon the forbidden region forcefully and, at the same time, sensibly. Other times, especially in the field of derivative-free optimization, algorithms include heuristic rules to avoid forbidden regions.

The algorithmic framework addressed in this paper is simple: we recognize that well-established algorithms exist for solving well-defined optimization problems, and we try to leverage their features as much as possible, instead of discarding them to look for radically different alternatives. The efficiency of this procedure lies in the modeler's ability to efficiently define the forbidden regions. Given that definition, the basic method consists of adopting a well-established algorithm—accepting that, occasionally, it may lead us to forbidden points—and treating those forbidden points as if they were points where the objective function is very large, but without using their value (which does not even need to be calculated or exist) for future computations. It is important to highlight that a strong motivation for employing forbidden region techniques is that objective functions can be very expensive to evaluate and, therefore, it is important to have criteria to avoid evaluating them when their evaluation promises to be useless.

Standard unconstrained minimization algorithms proceed, at each iteration, by calculating trial points that are accepted or rejected according to the value of the objective function. Even when rejected, these trial points contribute to the selection of subsequent trial points because they are frequently used in interpolation procedures that replace (simpler) backtracking steps. However, often before computing the first trial point in an iteration, the user possesses prior knowledge of the problem that indicates which regions are impossible or highly unlikely to contain a solution. Under these circumstances, and especially if the function evaluation is expensive, it can be advantageous to summarily reject unlikely trial points, avoiding useless searches on the boundary between the possible and the impossible.

*This work was supported by PRONEX-Optimization (PRONEX - CNPq / FAPERJ E-26 / 171.510/2006 - APQ1), FAPESP (Grants 2006/53768-0 and 2004/15635-2) and CNPq.

†Department of Applied Mathematics, IMECC-UNICAMP, University of Campinas, CP 6065, 13081-970 Campinas SP, Brazil. e-mail: fabio@ime.unicamp.br, martinez@ime.unicamp.br

This point of view is closely related to the classical notion of a trust region. Indeed, a trust region can be considered a region outside of which the occurrence of a solution is unlikely. If we possess a surrogate model of the objective function, it makes sense to minimize the model within the trust region to produce a trial point where evaluating the true function is worthwhile. However, an exhaustive search within the trust region to find a minimizer of the surrogate model can be expensive and inefficient. The results presented in this paper correspond to the case where the search within the trust region is reasonably exhaustive only if a solution promises to lie in the interior of the trust region, but not on its boundary.

Consider the problem

$$\text{Minimize } f(x) \text{ subject to } x \in I\!R^n \text{ and } x \notin P, \tag{1}$$ {problem}

where $P \subset I\!R^n$ is closed.
We assume that $\nabla f(x)$ exists and is continuous for all $x \in I\!R^n - P$. We denote $g(x) = \nabla f(x)$.

Algorithm 2.1.

Let $\theta \in (0,1)$, $\alpha \in (0, 1/2)$, and $\beta > 0$ and $\delta > 0$ be algorithmic parameters. Let $x^0 \in I\!R^n - P$ be the initial approximation.

Step 0. Initialize $k \leftarrow 0$.

Step 1. If $\|g(x^k\| = 0$, finish the execution of the algorithm.

Step 2. Compute $d^k \in I\!R^n$ such that

$$g(x^k)^T d^k \le -\theta \|d^k\|_2 \|g(x^k)\|_2 \text{ and } \delta \ge \|d^k\| \ge \beta \|g(x^k)\|. \tag{2}$$ {conds}

Step 3. Define

$$t_{first,k} = 1 \tag{3}$$ {tuno}

or

$$t_{first,k} = \sup\{t \in [0,1] \mid x^k + t d^k \notin P\}. \tag{4}$$ {tsup}

Set $t \leftarrow t_{first}, k$.

Set $\nu = 1$.

Step 4. If

$$x^k + t d^k \notin P \tag{5}$$ {notin}

and

$$f(x^k + t d^k) \le f(x^k) + \alpha t g(x^k)^T d^k \tag{6}$$ {armijo}

define $t_k = t$, $x^{k+1} = x^k + t_k d^k$, and $\nu_k = \nu$. Set $k \leftarrow k + 1$, and go to Step 1.

Step 5. Update $\nu \leftarrow \nu + 1$. Choose $t_{new} \in [0.1t, 0.9t]$, set $t \leftarrow t_{new}$, and go to Step 4.

Remark

Note that, under the choice (4), $x^k + t_{first,k} d^k$ may belong to P or not. If $t_{first,k} < 1$ we have that $x^k + t_{first,k} d^k \in P$, because, by the definition (4), there are elements of P of the form $x^k + t d^k$ for t arbitrarily close to $t_{first,k}$ and P is closed. But, if $t_{first,k} = 1$, possibilities ($x^k + t_{first}, k d^k \in P$ and $x^k + t_{first,k} d^k \notin P$) remain.

Lemma 2.1. *Every iteration of Algorithm 2.1 is well defined.*

Proof. By definition $x^0 \notin P$. Then, $g(x^k)$ exists and the algorithm stops at x^k if and only if $\|g(x^k)\| = 0$.

Assume that $x^k \notin P$ has been computed and $\|g(x^k)\| \neq 0$. Then, by (2),

$$g(x^k)^T d^k < 0. \tag{7}$$ {con}

We will prove that, after a finite number of inner steps, we can find $t > 0$ such that $x^k + t d^k \notin P$ and (6) takes place.

Since $x^k \in I\!R^n - P$ and $I\!R^n - P$ is open, there exists $\delta > 0$ such that $x \in I\!R^n - P$ whenever $\|x - x^k\| \leq \delta$.

Then, there exists $\bar{t} > 0$ such that $\|(x^k + td^k) - x^k\| \leq \delta$ for all $t \in [0, \bar{t}]$. But, after a final number of reductions of t at Step 4 we have that $t \leq \bar{t}$, so the condition (5) takes place for all $t \leq \bar{t}$. Then, by (7), the definition of directional derivative and the continuity of the gradient, we have that

$$\lim_{t \to 0} \frac{f(x^k + td^k) - f(x^k)}{t} = g(x^k)^T d^k < 0$$

So, since $\alpha < 1$, for t small enough we have that

$$\frac{f(x^k + td^k) - f(x^k)}{tg(x^k)^T d^k} \geq \alpha.$$

Therefore, (6) holds for t small enough. So, after a finite number of reductions of t, the acceptance condition (6) takes place. This completes the proof. □

Lemma 2.2 *Assume that for infinitely many indices $k \in K \subset I\!N$, $x^k + t_{first,k} d^k$ is chosen by (4) and belongs to P. Suppose that there exists ν_{max} such that $\nu_k \leq \nu_{max}$ for all $k \in K$ and $\lim_{k \to \infty} \|t_k d^k\| = 0$. Let x^* be a limit point of $\{x^k\}_{k \in K}$. Then $x^* \in P$.*

Proof. By the definition of the algorithm, $t_k \geq 0.1^{\nu_k} t_{first,k}$. Therefore, if $\|t_k d^k\|$ tends to zero and ν_k is bounded, we have that $\lim_{k \in K} \|t_{first,k} d^k\| = 0$. Now,

$$\|x^k + t_{first,k} d^k - x^*\| \leq \|x^k - x^*\| + \|t_{first,k} d^k\|,$$

So,

$$\lim_{k \in K} x^k + t_{first,k} d^k = x^*.$$

Since P is closed and $x^k + t_{first,k} d^k \in P$, this implies that $x^* \in P$. □

Theorem 2.1. *If x^* is a limit point of the sequence generated by Algorithm 2.1, we have that $x^* \in P$ or $\|g(x^*)\| = 0$.*

Proof. let $K \subset I\!N$ be an infinite sequence of indices such that

$$\lim_{k \in K} x^k = x^*. \tag{8}$$ {limK}

If $x^* \in P$ there is nothing to prove. So, from now on we assume that

$$x^* \notin P \text{ and } \|g(x^*)\| > 0. \tag{9}$$ {absu}

We will prove that (9) is impossible.

Let us consider two cases:

Case 1: $\lim_{k \in K} \|t_k d^k\| = 0$.

Case 2: There exists a subsequence $K_1 \subset K$ and $c_1 > 0$ such that $\|t_k d^k\| > c_1$ for all $k \in K_1$.

Clearly, Case 2 is the negation of Case 1.

Assume that Case 1 takes place.

Suppose, for a moment, that for infinitely many iterations $k \in K_1 \subset K$, the initial trial step $t_{first,k}$ has been chosen according to (4) and, in addition, t_k was the accepted step at the first choice of t after

the initial choice. In other words, $\nu_k = 2$ for all $k \in K_1$. Then, by Lemma 2.1, we have that that $x^* \in P$. Therefore, this possibility can be discarded.

Therefore, for $k \in K$ large enough, the choice of t_k has been preceeded by a choice of $t'_k \leq 10t_k$ such that (6) failed to hold when $t = t'_k$. Moreover, since $\lim_{k\in K} \|t_k d^k\| = 0$ we have that

$$\lim_{k\in K} \|t'_k d^k\| = 0. \tag{10}$$ {liteli}

So, for $k \in K$ large enough,

$$f(x^k + t'_k d^k) > f(x^k) + \alpha t'_k g(x^k)^T d^k \tag{11}$$ {noarm}

and

By (11) and the Mean Value Theorem, for all $k \in K$ large enough, there exists $\xi_k \in [0, 1]$ such that

$$g(x^k + \xi_k t'_k d^k)^T d^k > \alpha g(x^k)^T d^k. \tag{12}$$ {tvm}

By (10), dividing both members of (12) by $\|d^k\|$ and taking limits for $k \in K$ at a convergent subsequence of $d^k/\|d^k\|$ we get:

$$g(x^*)^T d \geq \alpha g(x^*)^T d \tag{13}$$ {thico}

where $\|d\| = 1$ and d es the limit of $d^k/\|d^k\|$ for a convergent subsequence. Since $0 < \alpha < 1$ This implies that

$$g(x^*)^T d \geq 0. \tag{14}$$ {thic}

But, by (2), we have that

$$g(x^k)^T d^k \leq -\theta \|g(x^k)\|_2 \|d^k\|_2,$$

So,

$$g(x^*)^T d \leq -\theta \|g(x^*)\|_2 < 0.$$

This contradicts (14). This contradiction comes from assuming that Case 1 takes place.

Therefore, we only need to analyze Case 2. By the definition of the algorithm we have that $x^k + t_k d^k \notin P$. Moreover, by (6),

$$f(x^k + t_k d^k) \leq f(x^k) + \alpha t_k g(x^k)^T d^k.$$

So, by (2),

$$f(x^k + t_k d^k) \leq f(x^k) - \alpha\theta t_k \|g(x^k)\|_2 \|d^k\|_2.$$

So, under the assumption that $\|g(x^*)\| > 0$, for $k \in K$ large enough we have that

$$f(x^k + t_k d^k) \leq f(x^k) - \alpha\theta cc_1. \tag{15}$$ {abajo}

Since $f(x^{k+1}) \leq f(x^k)$ for all $k \in I\!N$, (15) implies that $\lim f(x^k) = -\infty$, we contradicts the continuity of f at x^*. So, Case 2 is impossible too. Since both possibilities are impossible, it turns out that the assumption $\|g(x^*)\| > 0$ is impossible too. □

Pular para a Section 3.

Assumption 1 states that, in the proximity of any forbidden point, the function value is large.

Assumption 1. *For all $x \in P$ there exists $\delta = \delta(x) > 0$ such that*

$$[z \notin P \text{ and } \|z - x\| \leq \delta] \Rightarrow [f(z) \geq f(x^0)].$$

Theorem ??.2*Suppose that, at Step 3 of Algorithm* **??***.1, instead of $t \leftarrow 1$, we choose*

$$t \leftarrow \min\{t' \in [0, 1] \text{ such that } x + t' d^k \in P\}. \tag{16}$$ {wecho}

Then, each iteration of the algorithm remains well defined and the thesis of Theorem ??.1 also holds.

Theorem 2.2. *x^* is a limit point of the sequence generated by Algorithm 2.1, and Assumption 1 holds, then $\|g(x^*)\| = 0$.*

Proof. By (6) the sequence $\{f(x^k)\}$ is monotonically decreasing. Therefore, x^k cannot enter into the vicinity of a forbidden point. □

Remark: If you are solving a problem where the feasible set is a box and you are sure that, in the vicinity of the boundary of the box the function values are large, you do not need to employ box-constraint variations of an unconstrained algorithm and you can proceed only with backtracking. However, in these cases it is important to use single backtracking, that does not take into account the functional value, instead of parabolic interpolations.

Theorem 2.3. *Assume that f is continuous onto $\mathbb{R}^n$ and $f(x) > f(x^0)$ for all $x \in P$. Suppose that x^* is a limit point of the sequence generated by Algorithm 2.1. Then, $\|g(x^*)\| = 0$.*

Proof. It is enough to prove that Assumption 1 holds. In fact, for all $x \in P$ and $f(x) > f(x^0)$ we have that there exists $\delta > 0$ such that $f(z) \geq f(x^0)$ whenever $\|z - x^*\| \leq \delta$. □

Theorem 2.4. *Assume that f is continuous onto $\mathbb{R}^n$ and there exists $k \in \mathbb{N}$ such that $f(x) > f(x^k)$ for all $x \in P$. Suppose that x^* is a limit point of the sequence generated by Algorithm 2.1. Then, $\|g(x^*)\| = 0$.*

Proof. It is enough to prove that Assumption 1 holds. In fact, for all $x \in P$ and $f(x) > f(x^k)$ we have that there exists $\delta > 0$ such that $f(z) \geq f(x^k)$ whenever $\|z - x^*\| \leq \delta$. □

Theorem 2.5. *Assume that f is continuous onto $\mathbb{R}^n$ and $x^* \in P$ is a limit point of the sequence generated by Algorithm 2.1. Then, there exists an infinite number of indices k such that $f(x^*) < f(x^k)$.*

Proof. Directly from Theorem 2.4. □

3 Dynamic Forbidden Regions

{dina}

Assume that you have a problem in which the function is very expensive to evaluate. Moreover, there exists a large "yellow" region where the function is not defined, but you don't know exactly where this yellow region is, nothwithstanding some knowledge of its possible localization.

You start your iterative process from a nice point x^0 and you decide, in principle, that the forbidden region P is $\|x - x^0\| \geq \delta$, for some given δ.

You apply Algorithm 2.1 with the choice (4). According to Theorem 2.1, the algorithm "converges" to a point such that $\nabla f(x^*) = 0$ or "converges" to a boundary point, where $\|x - x^0\| = \delta$.

In the first case you are done. Stop.

In the second case, you adopt the final point as new iterate x^1. You build a new forbidden region and so on.

The question is: How do you know that you are in the second case? The only answer that I have to this question is that the current (internal) iterate of Algorithm 2.1 is very close to the forbidden boundary where $\|x - x^0\| = \delta$.

It is important that the backtracking process in this situation must be "conservative". (The trial point must be close to 0.1 and not to 0.9.) This is to avoid the possibility of convergence to the border very fast.

4 Vamos ver se isto funciona:

{vamosver}

Nosso objetivo é minimizar uma função que é uma soma de quadrados:

$$\text{Minimizar } f(x) = \frac{1}{2}\sum_{i-1}^{m} f_i(x)^2 \tag{17}$$ {lasoma}

ou seja

$$\text{Minimizar}\frac{1}{2}\|F(x)\|_2^2,$$

com

$$F(x) = (f_1(x), \cdots, f_m(x))^T.$$

Por Taylor, dado $a \in I\!R^n$, temos que

$$f_i(x) = f_i(a) + \nabla f_i(a)^T(x-a) + \frac{1}{2}(x-a)^T\nabla^2 f_i(a)(x-a) + O(\|x-a\|^3).$$

Portanto,

$$f_i(x)^2 = [f_i(a) + \nabla f_i(a)^T(x-a) + \frac{1}{2}(x-a)^T\nabla^2 f_i(a)(x-a)]^2 + f_i(a)O(\|x-a\|^3).$$

Logo,

$$f(x) = \sum_{i=1}^{m}[f_i(a) + \nabla f_i(a)^T(x-a) + \frac{1}{2}(x-a)^T\nabla^2 f_i(a)(x-a)]^2 + f(a)O(\|x-a\|^3). \tag{18}$$ {apro}

Esta fórmula induz de idea de minimizar, em cada iteração, a função $\sum_{i=1}^{m}[f_i(a) + \nabla f_i(a)^T(x-a) + \frac{1}{2}(x-a)^T\nabla^2 f_i(a)(x-a)]^2$. Mas isto é impraticável devido ao custo de $\nabla^2 f_i$.

Entretanto, podemos adotar a ideia de aproximar, via fórmulas quase-Newton, essas Hessianas. Isso leva à proposta de minimizar em cada iteração:

$$\text{Minimizar } \sum_{i=1}^{m}[f_i(a) + \nabla f_i(a)^T(x-a) + \frac{1}{2}(x-a)^T H_i(a)(x-a)]^2. \tag{19}$$ {thesu}

sujeita a uma região de confiança ou acrescentada de uma regularização.

(19) é a minimização de uma quártica, o que normalmente não é aconselhável. Mas no contexto de funções muito caras, esta ideia pode ser viável. Logo, para minimizar globalmente (19) não pouparíamos esforços.

Há muitas fórmulas quase-Newton. Por exemplo, as fórmulas BFGS e SR1, que são as seguintes:

$$H_+ = H + yy^T/s^Ty - (Hss^TH)/(s^THs) \tag{20}$$ {bfgs}

e

$$H_+ = H + (y-Hs)(y-Hs)^T/(y-Hs)^Ts. \tag{21}$$ {sr1}

Estas fórmulas são as que vão definir a direção d^k no Algoritmo 2.1, da seguinte maneira.

1. Inicializar $H_i = 0$ para todo $i = 1, \ldots, m$.
2. Se $k = 0$, $d^k = -\nabla f(x^k)$.
3. Se $k > 0$ definir $s \leftarrow x^k - x^{k-1}$, Para cada $i = 1, \ldots, m$ calcular $y = \nabla f_i(x^k) - \nabla f_i(x^{k-1})$. Para cada $i = 1, \ldots, m$ calcular H_+ usando (20) ou (21). Definir $H \leftarrow H_+$. Resolver, globalmente, (19) com $a = x^k$, obtendo x. Definir $d^k = x - x^k$. Verificar as condições (2). Se não se verificam, substituír d^k pela direção de $-\nabla f(x^k)$. Espero que se verifiquem sempre, em caso contrário tanto trabalho foi jogado no lixo.